AutoBayes brms-parity loop: Tier A review

Status at the Tier A boundary, prepared for Andrew

7 October 2026

This is the tier-boundary review the approved plan calls for: what landed, what parked, where the library knowingly differs from brms, how each claim was checked, and what needs your decision before Tier B starts. Section 8 is the part that needs you; the rest is the evidence behind it.

1 What "merged" means in this loop

Every feature is accepted by comparison with brms itself, never by sampling. For a model brms accepts, the oracle (`oracle/R/gen_fixture.R`) records, once, deterministically: the Stan data (`standata()`), the log density and its gradient at eight seeded points of the unconstrained space (`rstan::log_prob`, with and without the Jacobian), and `posterior_linpred`/`posterior_epred` at those points. That record is a frozen fixture. A feature merges when the AutoBayes model built from the same data reproduces it:

- G1 design: design matrices, response, trials, offsets and group coding equal `standata` to 10^{-12};
- G2 density: log density, its prior part and every gradient coordinate equal brms to 10^{-8} / 10^{-7} relative (relative to $\max(1, |a|, |b|)$, so at a log density near -120 the floor is about 10^{-6} absolute);
- G3 derived quantities and G4 predictions to 10^{-9}; G5 the JAX density equals the PyTensor one; G6 `ab.default_priors()` reproduces brms's `default_prior()` table string for string (strict since A07);
- G7 the whole non-sampling suite locally, and the full fast suite in hosted CI on the pull request.

What this does *not* establish: that NUTS converges on these targets, or that intervals are calibrated. It establishes that the Python model *is* brms's target, coordinate by coordinate. Nothing in the gates samples: the only sampling in Tier A was the existing short sampling tests that hosted CI runs on every pull request, and the advisory 120-second smoke (G8) was never invoked.

2 What landed

Feature	PR	What a user can now write	Frozen brms cases it passes	Diverg.
A01 independent slopes	#29	`(1 + x [+ w] || g)`, any numeric slopes; `sd={"g": {"Intercept": ..., "x": ...}}` (your API decision)	`gauss_slope_independent`, `_two_groups`, `_two_slopes`, `bernoulli_slope_categorical`, `poisson_slope_independent`	D8
A02 correlated slopes	#35	`(1 + x | g)` with `cor={"g": ab.LKJ(eta)}`; L sampled in Stan's own Cholesky-correlation coordinates	`gauss_slope_correlated`	D20, D21
A03 grouping syntax	#31	`(1 | g:h)`, `(1 | g/h)`, brms's `g_h` labels	`gauss_nested_colon`, `_slash`	D11, D12
A04 interactions	#34	`x:w`, `z:h` (two-way, main effects present)	`gauss_interaction_numeric_numeric`, `_categorical_categorical`	D14, D15
A05 transforms	#34	`log(x)`, `exp(x)`, `sqrt(x)`, `I(x^k)`, `I(x * k)`, `I(x + k)`, `I(x - k)`	`gauss_transforms`	D16
A06 offset	#31	`offset(col)`, `offset(log(col))`	`poisson_offset`	D10
A08 negative binomial	#30	`ab.negbinomial`, `Priors(shape=...)`	`negbinomial_varint`	–
A09 Gamma	#30	`ab.gamma`, log link	`gamma_varint`	D9
A10 lognormal	#32	`ab.lognormal`; `kind="mean"` is brms's $e^{\mu+\sigma^2/2}$	`lognormal_varint`	–
A11 Beta	#32	`ab.beta`, `Priors(phi=...)`	`beta_varint`	D13
A12 links	#36	`link="probit"` for Bernoulli and binomial; cloglog parked (Section 3)	`bernoulli_probit`, `binomial_probit`	D17–D19
A13 learned ν	#36	`ab.student(..., nu=None)` with `Priors(nu=...)`, truncated at 1	`student_learned_nu`	D22
A07 default priors	#37	`ab.default_priors(formula, data, family=...)`: brms's defaults on request, data-dependent intercept location and scales reproduced exactly	G6 on all 38 ungated cases; 5 new cases frozen without priors	D1, D2, D23

Infrastructure behind these (Phase 0, all merged): #22 an automatic `sampling` marker so the deterministic suite runs alone; #23 the oracle and its first 14 fixtures; #24 the loop's rules, ledger and gate tools; #25 a family registry (no `family ==` branches remain; a lint test forbids new ones); #27 an explicit term model (`PopTerm`, `GroupTerm`, per-coefficient group blocks), which is what let slopes, grouping syntax and interactions land as small additions rather than rewrites. Fixtures for every feature were frozen before any implementation existed (#26, #28, #33) and ran as strict expected failures until their branch removed the gate.

How close. The gates only say "within tolerance". The table below says by how much: for every ungated case, the worst relative error against brms over the eight points (log density with and without the Jacobian; every gradient coordinate; linear predictor and expected value on fitted and new rows, with and without group effects). Most sit at 10^{-16}–10^{-15}, i.e. the same number up to rounding. The exceptions follow the special functions involved: the cases with Student-t-type

densities (the Student-t likelihood; half-Student-t and half-Cauchy priors, which includes every case run under brms's default priors) have log-density differences of 2×10^{-11} to 2.3×10^{-10}, and the families whose gradients involve digamma (Beta, Gamma, negative-binomial shape, learned ν) have gradient differences of 4×10^{-11} to 9.4×10^{-10}; for the last, the A13 work traced it to PyTensor's C digamma (about 2×10^{-10} accurate, where JAX's agrees with brms to 3×10^{-15}). I have not traced the Student-t density difference further. Even the worst cases are 40, 100 and 10^6 times inside the 10^{-8}, 10^{-7} and 10^{-9} tolerances. (Measured on the A07 branch, which contains everything in Tier A; 38 cases.)

case	family	coords	log density	gradient	predictions
`bernoulli_interaction_two_groups`	bernoulli	18	3.0e-16	8.9e-16	6.3e-16
`bernoulli_probit`	bernoulli	9	3.3e-16	4.5e-15	3.6e-16
`bernoulli_slope_categorical`	bernoulli	18	2.8e-16	1.0e-15	4.4e-16
`bernoulli_slope_default_priors`	bernoulli	17	1.0e-10	6.7e-16	4.0e-16
`bernoulli_varint`	bernoulli	9	3.2e-16	1.2e-15	2.2e-16
`beta_varint`	beta	10	9.6e-16	2.7e-10	4.1e-16
`binomial_poponly_categorical`	binomial	4	6.7e-16	5.0e-16	4.6e-16
`binomial_probit`	binomial	9	9.4e-15	2.3e-13	1.0e-15
`binomial_varint`	binomial	9	6.5e-16	1.3e-15	4.4e-16
`gamma_varint`	gamma	10	7.3e-16	3.0e-10	4.6e-16
`gauss_categorical`	gaussian	12	4.6e-11	1.4e-15	4.4e-16
`gauss_interaction_categorical_categorical`	gaussian	13	2.7e-16	1.3e-15	3.3e-16
`gauss_interaction_numeric_numeric`	gaussian	12	4.7e-16	3.7e-15	6.4e-16
`gauss_interaction_two_groups`	gaussian	19	3.6e-16	4.4e-15	3.3e-16
`gauss_intercept_only`	gaussian	9	4.0e-16	7.3e-16	0.0e+00
`gauss_nested_colon`	gaussian	30	2.9e-16	1.7e-15	4.4e-16
`gauss_nested_slash`	gaussian	31	2.7e-16	1.4e-15	4.4e-16
`gauss_no_intercept`	gaussian	5	5.2e-16	4.0e-15	0.0e+00
`gauss_poponly`	gaussian	4	4.3e-16	7.7e-15	3.7e-16
`gauss_slope_correlated`	gaussian	18	3.8e-16	3.2e-15	5.6e-16
`gauss_slope_independent`	gaussian	17	2.1e-16	1.3e-15	4.4e-16
`gauss_slope_two_groups`	gaussian	23	2.1e-16	1.6e-14	8.9e-16
`gauss_slope_two_slopes`	gaussian	25	4.5e-16	7.2e-15	7.8e-16
`gauss_transforms`	gaussian	11	5.9e-16	4.6e-15	3.9e-16
`gauss_varint`	gaussian	10	3.5e-16	9.1e-16	2.3e-16
`gauss_varint_default_priors`	gaussian	10	3.2e-11	1.5e-14	2.3e-16
`gauss_varint_uncentered`	gaussian	10	3.3e-16	3.4e-15	2.2e-16
`lognormal_varint`	lognormal	10	3.7e-16	7.4e-16	9.7e-16
`negbinomial_default_priors`	negbinomial	10	2.0e-11	1.6e-10	4.4e-16
`negbinomial_varint`	negbinomial	10	6.7e-16	3.8e-11	4.9e-16
`poisson_categorical_default_priors`	poisson	11	2.1e-11	1.0e-15	5.6e-16
`poisson_offset`	poisson	9	4.9e-16	2.6e-15	8.8e-16
`poisson_poponly_categorical`	poisson	4	3.0e-16	1.0e-15	4.3e-16
`poisson_slope_independent`	poisson	16	2.7e-16	2.3e-15	7.9e-16
`poisson_varint`	poisson	9	6.1e-16	1.1e-15	4.3e-16
`student_default_priors`	student	11	2.2e-11	1.5e-10	3.8e-16
`student_learned_nu`	student	11	3.8e-16	9.4e-10	3.1e-16
`student_varint`	student	10	2.3e-10	2.5e-15	2.2e-16

3 The one park: brms's cloglog link

The probit link landed for Bernoulli and binomial models. The complementary log-log link did not, and the reason is a defect in brms's arithmetic rather than in ours, so it is worth reading through once.

With $\theta = 1 - \exp(-e^{\eta})$, a row with $y = 0$ contributes $\log(1 - \theta) = -e^{\eta}$ to the log density. Its derivative in η is exactly $-e^{\eta}$. Stan instead evaluates `bernoulli_lpmf(0 | inv_cloglog(eta))`, which forms $1 - \theta$ *by subtraction* from a stored θ.

The numbers at the frozen point. At the sixth of the eight seeded points of `bernoulli_cloglog` (index 5 in the fixture), one $y = 0$ row has $\eta = 3.008$. Then $e^{\eta} = 20.25$ and $1 - \theta = e^{-20.25} \approx 1.6 \times 10^{-9}$, while $\theta \approx 1 - 1.6 \times 10^{-9}$ is stored to about 1.1×10^{-16} absolute. Subtracting leaves $1 - \theta$ with up to $1.1 \times 10^{-16}/1.6 \times 10^{-9} \approx 7 \times 10^{-8}$ relative error, and that row's gradient term is multiplied by $e^{\eta} \approx 20$. The fixture records the coefficient gradient as -1.0055917217903116; the exact value (50-digit arithmetic from the fixture's own data and parameters) is -1.0055921064960804, a relative error of 3.8×10^{-7}, above the 10^{-7} tolerance. A float64 imitation of Stan's subtraction reproduces brms's number to 7×10^{-15}, and regenerating the fixture from R reproduces it exactly, so this is deterministic brms behaviour, not a fixture fault. AutoBayes's stable form ($\log(1 - \theta) = -e^{\eta}$ directly) is within 4.6×10^{-15} of exact at that point; at the other seven points the two agree to 2.7×10^{-13}.

Options. (a) Keep it parked: `link="cloglog"` is refused with this reason, the stable form stays in `likelihoods.py` behind tests (recommended). (b) Reproduce brms's subtraction so the gate passes: that copies a loss of accuracy into the library on purpose. (c) Accept cloglog against a closed-form reference instead of brms for this one link: possible, but it is the first feature that would not be oracle-accepted.

4 Where AutoBayes knowingly differs from brms

Every difference is logged in `docs/brms-parity/divergences.md`; fixtures were never edited to hide one. Most are deliberate restrictions of scope with a clear refusal message; a few are numerically better forms of the same density that agree with brms to within the tolerances on every frozen point.

	area	brms	AutoBayes	status
D1	priors	default priors for every class	every prior explicit; a class the model lacks is refused	kept; brms's defaults on request (A07)
D2	priors	intercept-only model needs no `b` prior	still asks for a vacuous `b` prior	resolved (A07): no `b` prior needed
D3	factor levels	R factor order (locale-dependent)	levels sorted as strings	handled: matched by content
D4	binomial	keeps zero-trial rows	drops and reports them	kept
D5	binomial means	`posterior_epred` is np	`kind="mean"` gives p and `expected` $= np$	documented
D6	new levels	resamples by default	refuses by default; two explicit policies	documented
D7	Bernoulli	`bernoulli_logit_lpmf`	gradient-stable softplus form	equal on fixtures
D8	slopes	slope without its population term allowed	slope must also be a population term	kept
D9	Gamma	inverse link by default	log link only	kept
D10	offset	any expression, several offsets	`offset(col)`, `offset(log(col))`, one	kept
D11	grouping	three or more columns combined	two columns	kept
D12	grouping	duplicate partitions; colliding `_` labels merge	both refused	kept
D13	Beta	$1-\mu$ by subtraction (0 beyond $\eta \approx 37$)	$1-\mu$ as expit$(-\eta)$	equal on fixture
D14	interactions	missing main effects, any order	two-way, both main effects	kept
D15	interactions	offset or transformed operands	main-effect columns only	kept
D16	transforms	any R expression	seven forms of one column	kept
D17	probit	Φ saturates beyond -37.5 / 8.25	stable $\log \Phi$	equal on fixtures
D18	cloglog	$1-\theta$ by subtraction	refused with the reason	parked
D19	other links	cauchit; links of other families	logit and probit for Bernoulli/binomial	kept
D20	correlations	correlation IDs, `gr()`	one matrix per term; IDs and `gr()` refused	kept
D21	correlations	a scalar per coefficient pair	whole matrix `cor|g` and factor `L|g`	names only
D22	Student-t ν	any prior, renormalised; default `gamma(2, 0.1)`	Gamma, Exponential, half-normal, half-Cauchy	kept
D23	default priors	a fixed Student-t ν still listed with `gamma(2, 0.1)`	no ν prior when ν is fixed	documented

"Kept" means a deliberate restriction with a refusal that names the construct and says what to write instead; "equal on fixtures" means a numerically safer form of the same density, within the tolerances on every frozen point.

5 A07: brms's default priors

Explicit priors remain the default: nothing is filled in silently (D1). A07 adds brms's defaults *on request*: `ab.default_priors(formula, data, family=...)` returns the `Priors` brms would use, each marked as a default in `describe()` and in fit records. The hard part is that several of brms's defaults depend on the data. For the intercept brms uses student_t$(3, m, s)$ with m the median of the response on the link scale and $s = \max(2.5, \text{MAD})$, both rounded and printed by R's own rules; for example `poisson_offset`'s table reads `student_t(3, -0.978004218445523, 2.5)`. These are transcribed from brms and R and reproduced exactly, so gate G6 now compares brms's `default_prior()` table with the library's row by row and string by string, and passes on all 38 ungated fixtures; a one-unit change in the 15th digit fails it (a harness test). Five new fixtures were frozen with no priors at all on the brms side, so G2 also checks the densities *under* the defaults: a flat `b`, the data-dependent intercept, the bounded half-Student-t scales, `inv_gamma(0.4, 0.3)` for the negative binomial shape, `lkj(1)`, and `gamma(2, 0.1)` above 1 for a learned ν.

Beyond the fixtures (scratch checks, not committed), the transcription matched R on about 340,000 values of `round`, `as.character`, `median`, `mad`, `mean` and `log`, and a differential test against `brms::default_prior()` matched 329 of 330 synthetic configurations (the other has a constant Bernoulli response, which the constructors refuse). That test is what found brms's rule for naming an interaction (`x:z + z + x` names the term `zq:x`). Side effect: an intercept-only formula no longer needs a vacuous `b` prior (D2 resolved). Found in passing and not yet fixed: the order of two lines in a learned-ν model's `describe()` depends on Python's hash seed (PyMC orders those two value variables by hash); the target identity is unaffected.

6 How the claims were checked

- Each feature: the gate tool on the exact commit (immutability of fixtures and tolerances, the oracle tests, the full non-sampling suite), recorded with the commit SHA in the ledger; then both hosted CI runs on the pull request; then squash-merge as andrew-dou. Changes to `tests/oracle/adapter.py` and `gates.py` were read in review for weakening every time; all were extensions (new parameter kinds), and the correlated-slope change replaced a blanket refusal with an agreement check that is itself tested both ways.
- Each refactor (registry, term model) and each feature: every previously ungated oracle-case model's target identity, `describe()` text, names and coefficients were snapshotted before the change and compared after; all identical in every case.
- Reproducibility at the boundary: `oracle/regen.py check -all` regenerated the 43 frozen fixtures on `main` from R and compared them with the committed ones; the six frozen since were checked the same way: all 49 identical.

Limits I know of. Correlated slopes are checked against brms for two coefficients only (the one fixture); three coefficients are checked against closed forms, and Stan's LKJ normalising constant for three or more coefficients with $\eta \neq 1$ is not independently confirmed. The special-function differences above (at most 9.4×10^{-10}) are inside tolerance today but would be the first to matter if a tolerance were ever tightened.

7 Process and cost

Each pair of features was implemented by one agent working in its own git worktree from a written brief, then gated, reviewed and merged by me (squash, as andrew-dou, no AI attribution). Token counts reported by the ten agents that completed: P3 0.31M, P4 0.33M, A01 0.45M, A08+A09 0.39M, A06+A03 0.50M, A10+A11 0.46M, A04+A05 0.62M, A02 0.36M, A12+A13 0.58M, A07 0.59M, about 4.6M in total. Not included: four agents cut off by limits before reporting a count, and my own orchestration. Each agent ran 34–83 minutes; the non-sampling suite (about 1,300 tests) takes 6–17 minutes on this machine depending on load.

Two interruptions. On 6 October the included usage ran out; state was persisted and the work resumed after the reset. On 7 October the Fable 5.1 monthly spend limit stopped two agents mid-work; their uncommitted work was backed up, carried onto the current `main`, and finished after you switched the session to Opus 5.5, with agents pinned to Opus.

8 Decisions before Tier B

1. **The cloglog park** (Section 3). I recommend keeping it parked: option (a). Options (b) and (c) are both available if you want the link.
2. **API changes already merged that you may want to look at.** None breaks an existing call, but they change what some calls mean:
   - `ab.student`'s `nu` now defaults to `None`, meaning "learn it", which requires `Priors(nu=...)`. A call without `nu` used to fail as a missing argument; it now fails with a message saying what to pass.
   - `(1 + x | g)` now means correlated effects (brms's meaning); it used to be refused. `(1 || g)` is accepted as `(1 | g)`.
   - `Priors` gained `shape`, `phi`, `cor` and `nu`, all after the four original fields, so positional calls keep their meaning. Bernoulli and binomial models gained `link=`.
3. **Tier B.** Ten brms fixtures are already frozen for its six features: distributional `sigma z`, `cens()`/`trunc()`/`weights()`, `mo()`, `s()`, cumulative and categorical families, zero-inflated and hurdle Poisson. At Tier A's rate that is roughly 1.5–2M agent tokens over three two-feature agents. I expect `s()` to park: brms builds the spline basis with mgcv, which nothing in the pinned stack reproduces, so it would mean porting mgcv's thin-plate basis (my estimate; not yet attempted). I would start with the addition terms and the zero-inflated/hurdle families, which reuse the most existing machinery. Go ahead?
4. **Cleanup.** Sixteen local worktrees under `AutoBayes-worktrees/` belong to merged or closed branches: `parity-p1`, `-p2`, `-p3`, `-p4`, `-a01`, `-a01-fixtures`, `-tierA-fixtures`, `-tierB-fixtures`, `-fam1`, `-fam2`, `-fam3`, `-form1`, `-form2`, `-a02`, `-a07`, and `m3b1-varying-slope` (the closed PR #8). Their branches stay on GitHub. `group-exclusion` holds the virtual environment every worktree uses and stays; `parity-loopstate` is the merge-queue checkout of `main`; your original checkout `projects/AutoBayes` is untouched. I will remove the sixteen only on your yes.

Also in this pass: a documentation pull request that adds a current-status section to the README and SKILL.md (they still describe slopes, `g/h`, interactions and transforms as refused) without removing the older milestone text, and corrects two stale divergence references.